

Instituto Tecnológico Autónomo de México
Computo Paralelo y En La Nube

Proyecto 1

K-means Paralelo con OpenMP

Integrantes:

Andres Samano
Alejandro Salas

Marzo 2026

1. Introduccion

Este proyecto implementa el algoritmo de clustering K-means en su version serial y en su version paralelizada utilizando OpenMP, con el objetivo de evaluar el speedup obtenido al distribuir el computo entre multiples hilos en una maquina de memoria compartida.

El algoritmo K-means es ampliamente utilizado en mineria de datos y aprendizaje automatico para agrupar conjuntos de datos en K clusters basandose en la distancia euclidiana. Su naturaleza iterativa y el elevado volumen de operaciones matematicas lo convierten en un candidato ideal para la paralelizacion.

Se implementaron versiones para datos en dos dimensiones (x, y) y tres dimensiones (x, y, z), con datasets de hasta un millon de puntos, evaluando el desempeno con distintas cantidades de hilos OpenMP.

2. Descripcion del Algoritmo K-means

K-means es un algoritmo de clustering no supervisado que agrupa N puntos en K grupos minimizando la suma de distancias al cuadrado entre cada punto y el centroide de su cluster asignado. El algoritmo funciona de la siguiente manera:

- Inicializacion: Se seleccionan K puntos aleatorios del dataset como centroides iniciales (semilla fija=42 para reproducibilidad).
- Asignacion: Cada punto se asigna al centroide mas cercano, calculando la distancia euclidiana al cuadrado (sin raiz cuadrada para mayor eficiencia, ya que solo se comparan distancias).
- Actualizacion: Cada centroide se recalcula como el promedio de coordenadas de todos los puntos asignados a el.
- Convergencia: Los pasos 2 y 3 se repiten hasta que ningun punto cambia de cluster, o se alcanzan 100 iteraciones maximas.

3. Descripcion delCodigo

El proyecto esta implementado en C++ y consta de cuatro archivos fuente:

Archivo	Descripcion
kmeans_serial_2d.cpp	Version serial para datos en 2 dimensiones (x, y)

kmeans_serial_3d.cpp	Version serial para datos en 3 dimensiones (x, y, z)
kmeans_paralelo_2d.cpp	Version paralela con OpenMP para datos 2D
kmeans_paralelo_3d.cpp	Version paralela con OpenMP para datos 3D

Cada archivo implementa las mismas estructuras: Punto (coordenadas + etiqueta de cluster) y Centroide. Las funciones principales son: leerCSV(), guardarCSV(), distancia2() (distancia euclidiana al cuadrado), inicializarCentroides() y kmeans(). Los programas reciben argumentos por linea de comandos: archivo de entrada, K, archivo de salida y (en version paralela) numero de hilos.

4. Estrategia de Paralelizacion con OpenMP

4.1 Asignacion de Puntos (Paso 1)

El loop de asignacion distribuye los puntos entre hilos usando:

```
#pragma omp parallel for schedule(static) shared(puntos, centroides)
reduction(||:cambio)
```

Cada hilo procesa un subconjunto contiguo de puntos de forma independiente. La clausula reduction(||:cambio) garantiza que la deteccion de convergencia sea correcta sin condiciones de carrera.

4.2 Calculo de Centroides (Paso 2)

El recalcu de centroides usa sumas locales por hilo para evitar condiciones de carrera. Cada hilo acumula sus propias sumas y conteos, y al finalizar los combina en las sumas globales con una seccion critica (#pragma omp critical). Este patron minimiza el tiempo en la seccion critica: cada hilo entra solo una vez por iteracion, no una vez por punto.

5. Descripcion del Experimento

5.1 Generacion de Datos

Los datos se generaron con make_blobs de scikit-learn, produciendo puntos aleatorios en K=3 clusters dentro del hipercubo unitario [0,1], con random_state=42 para reproducibilidad. Se generaron 14 archivos CSV: 7 tamanos para 2D y 7 para 3D.

5.2 Parametros

Parametro	Valores
Dimensiones	2D (x, y) y 3D (x, y, z)
Numero de puntos	100K 200K 300K 400K 600K 800K 1,000K
Clusters K	3
Hilos	1 6 (cores/2) 12 (cores) 24 (cores*2)
Repeticiones	10 por configuracion (se promedia el tiempo)
Max iteraciones K-means	100

El speedup se calcula como: **Speedup = Tiempo Serial / Tiempo Paralelo**. Valores mayores a 1 indican ganancia; menores a 1 indican que el overhead de hilos supera el beneficio.

6. Descripcion del Equipo

6.1 Hardware

Componente	Especificacion
Equipo	Razer Blade 15 Base Early 2021 (RZ09-0369)
Procesador	Intel Core i7-10750H @ 2.60 GHz
Nucleos fisicos	6 nucleos fisicos
Nucleos logicos	12 procesadores logicos (Hyper-Threading)
Memoria RAM	16 GB
Arquitectura	x64

6.2 Software

Componente	Version/Detalle
Sistema Operativo	Windows 10 Home (Build 19045)
Compilador	g++ MinGW-w64
Flags de compilacion	-O2 -fopenmp

API de paralelizacion	OpenMP
Lenguaje	C++
Python	3.10 (generacion de datos y graficas)
Librerias Python	scikit-learn, numpy, pandas, seaborn, matplotlib

7. Resultados y Analisis

7.1 Speedup vs Numero de Puntos — 2D

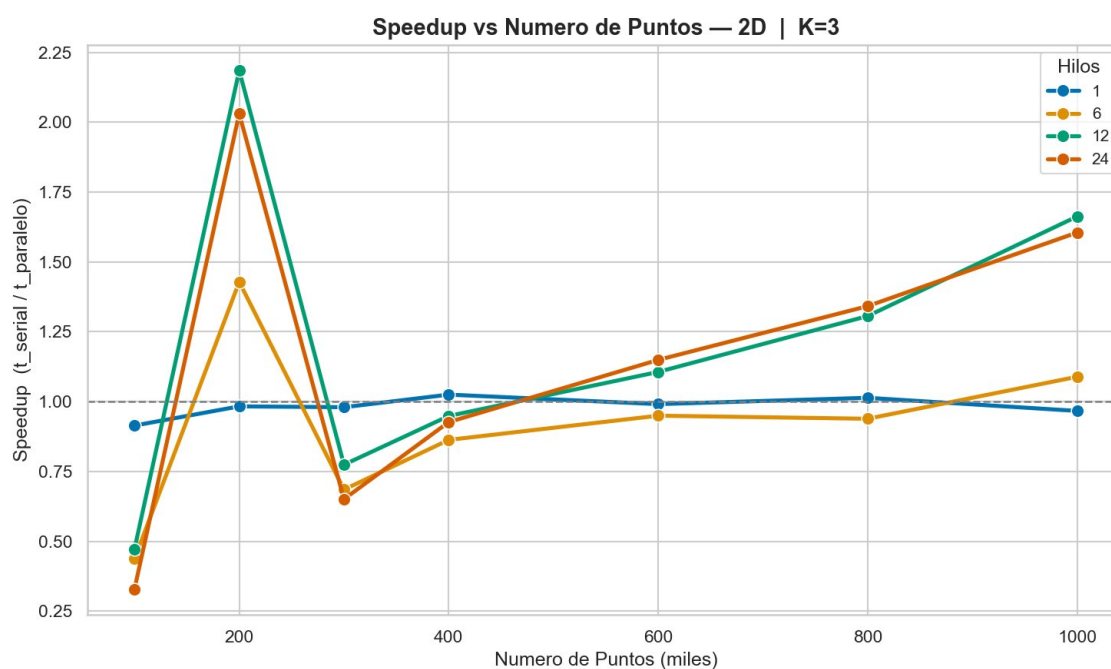


Figura 1. Speedup vs numero de puntos en 2D.

En 2D se observa un comportamiento irregular para datasets pequenos (100K-400K puntos), donde el speedup cae frecuentemente por debajo de 1. El overhead de crear y sincronizar hilos supera la ganancia cuando el trabajo por hilo es insuficiente. A partir de 800K puntos, el speedup se estabiliza por encima de 1 para 12 y 24 hilos, alcanzando un maximo de **1.66x** con 12 hilos en 1M de puntos.

7.2 Speedup vs Numero de Puntos — 3D

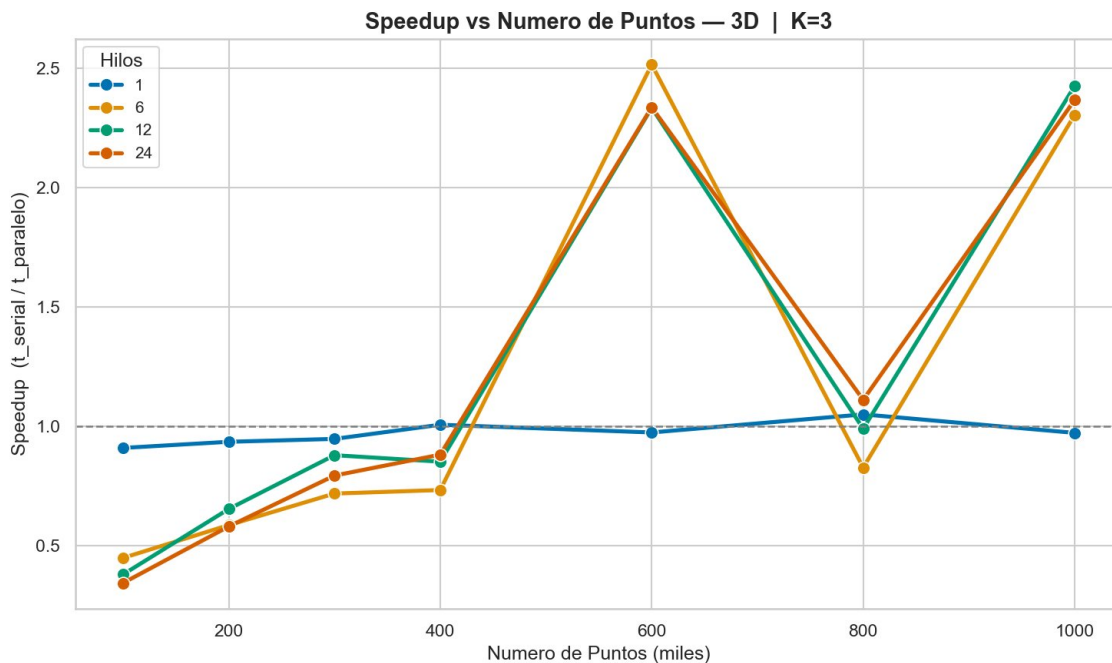


Figura 2. Speedup vs numero de puntos en 3D.

En 3D los datasets de 600K y 1M puntos requirieron mas iteraciones para converger, resultando en tiempos seriales mucho mayores (hasta 0.76s para 1M puntos 3D vs 0.034s para 2D). Este mayor trabajo beneficia la paralelizacion: se obtuvo el mejor speedup del experimento con **2.43x** usando 12 hilos en 1M puntos 3D.

7.3 Tiempo vs Numero de Puntos — 2D

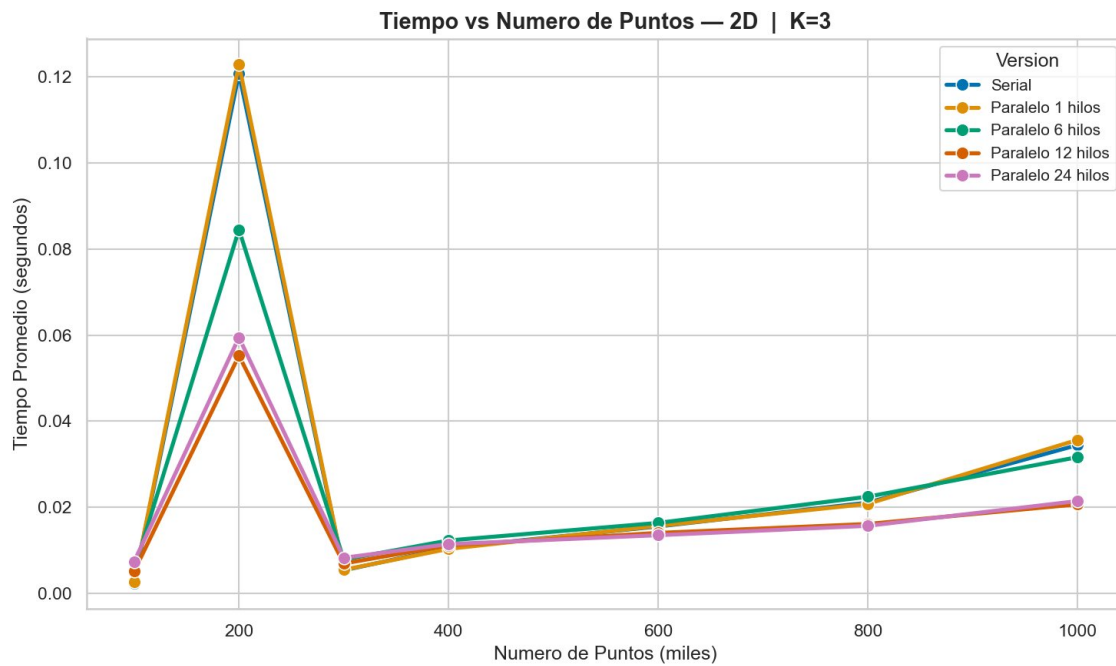


Figura 3. Tiempo de ejecucion vs numero de puntos en 2D.

En 2D los tiempos absolutos son bajos (milisegundos), por lo que las diferencias entre versiones son pequenas. Las lineas de 12 y 24 hilos muestran una separacion visible solo a partir de 800K puntos, confirmando que la paralelizacion solo es rentable con datasets grandes.

7.4 Tiempo vs Numero de Puntos — 3D

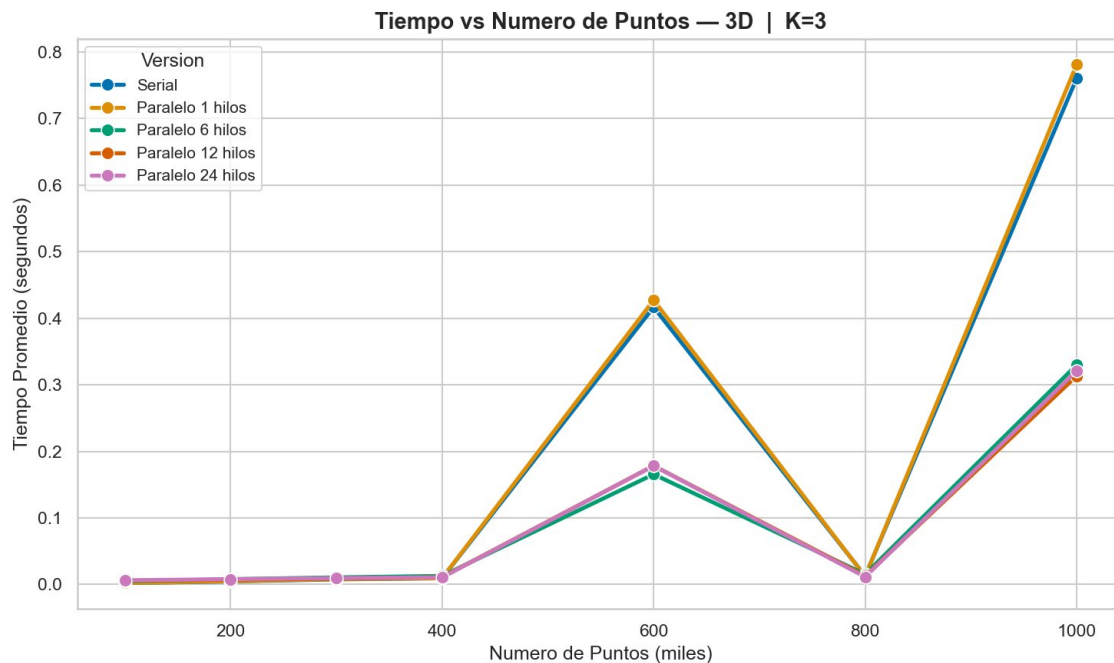


Figura 4. Tiempo de ejecucion vs numero de puntos en 3D.

En 3D las diferencias son notorias para 600K y 1M puntos. La version serial toma hasta 0.76s para 1M puntos, mientras que el paralelo con 12 hilos lo reduce a 0.31s. La linea del paralelo con 1 hilo sigue de cerca a la serial, confirmando que el overhead de OpenMP sin paralelismo real es minimo.

7.5 Speedup vs Numero de Hilos — 2D

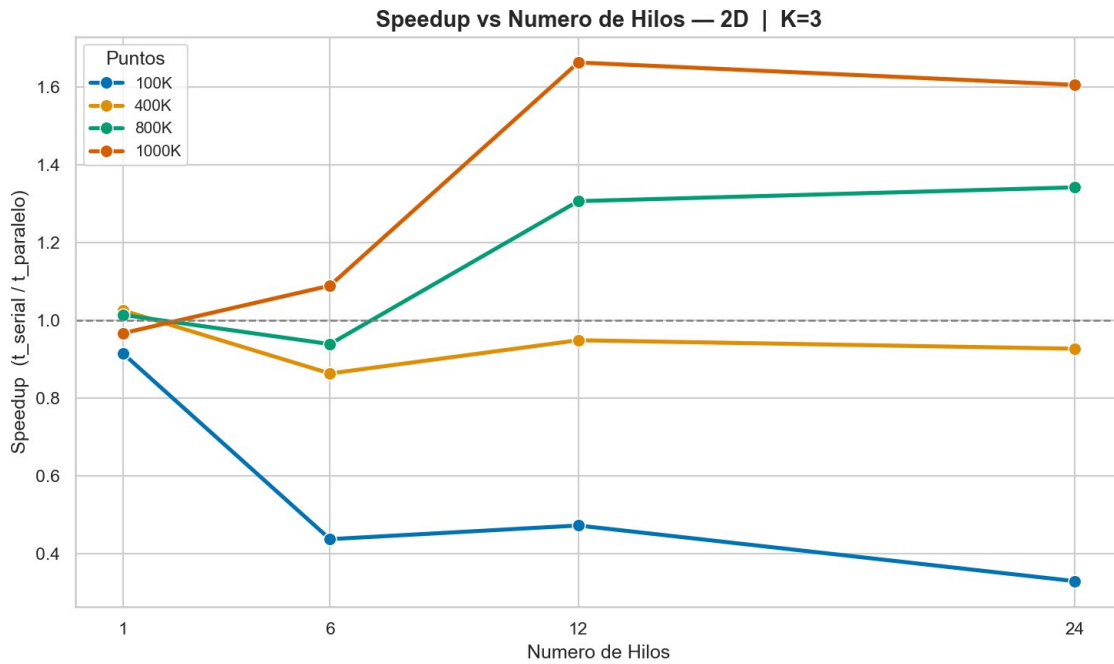


Figura 5. Speedup vs numero de hilos en 2D.

Para 100K puntos el speedup decrece con cada hilo adicional, confirmando que el tamaño es insuficiente. Para 1M puntos el speedup crece hasta 12 hilos (1.66x) y se estabiliza con 24 hilos, indicando que el número óptimo de hilos corresponde al número de procesadores lógicos de la máquina.

7.6 Speedup vs Numero de Hilos — 3D

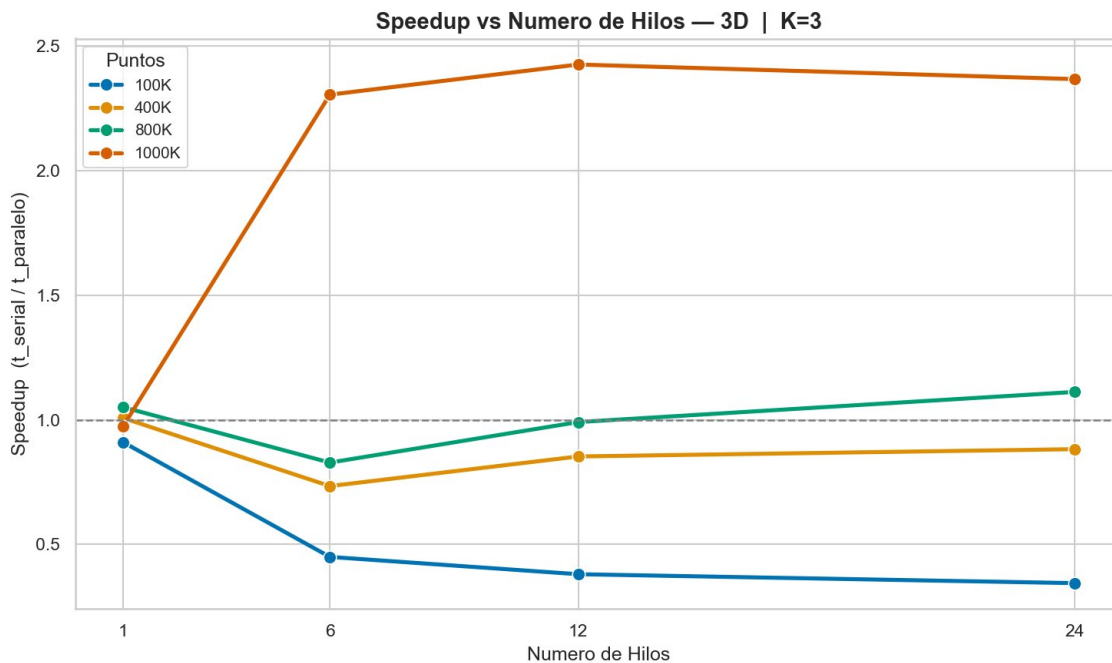


Figura 6. Speedup vs numero de hilos en 3D.

En 3D para 1M puntos el speedup alcanza 2.43x con 12 hilos y baja ligeramente a 2.37x con 24 hilos, sugiriendo que el hyperthreading no ofrece beneficio adicional para este tipo de computo. Para datasets pequenos el patron es identico al 2D: el overhead supera la ganancia.

8. Conclusiones

- Para datasets pequenos (100K-400K puntos) el overhead de hilos supera la ganancia. No se recomienda paralelizar K-means con OpenMP para estos volúmenes.
- Para datasets grandes (800K-1M puntos) la paralelización ofrece ganancias reales. El mejor resultado fue 2.43x en 3D con 1M puntos y 12 hilos.
- El número óptimo de hilos es 12 (igual al número de procesadores lógicos). Usar 24 hilos (hyperthreading) no mejora el desempeño para operaciones de memoria intensivas.
- Los datos 3D muestran mejores speedups que 2D porque requieren más computo por punto y en ciertos datasets convergen en más iteraciones, justificando mejor el paralelismo.
- La estrategia de sumas locales por hilo para el recálculo de centroides fue clave para evitar condiciones de carrera con mínimo impacto en el rendimiento.

Como trabajo futuro, seria interesante implementar K-means++ para mejor convergencia, explorar MPI para distribucion entre multiples maquinas, o paralelizacion en GPU con CUDA para datasets de mayor escala.